

# Conditionals

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - [www.cloudcontraptions.com](http://www.cloudcontraptions.com)

# if, else, else if

---

```
let score = 75;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 80) {
  console.log("Grade: B");
} else if (score >= 70) {
  console.log("Grade: C");
} else {
  console.log("Grade: D or below");
}
```

- The **first** condition that is `true` runs; the rest are skipped
- Order matters: put the narrowest test first

# Truthy and Falsy

---

- `if` checks whether a value is **truthy** or **falsy**
- Falsy values: `false`, `0`, `""`, `null`, `undefined`, `NaN`
- Everything else is **truthy**, including `[]` and `{}`

```
let name = "";
if (name) {
  console.log("Name is set.");
} else {
  console.log("Name is empty or missing."); // this runs
}
```

# Logical Operators

---

```
const age = 20, hasTicket = true;

age >= 18 && hasTicket; // true - AND: both must be true
age < 18 || hasTicket;  // true - OR: either will do
!hasTicket;             // false - NOT: flips it
```

- They **short-circuit**: `&&` stops at the first falsy, `||` at the first truthy

```
user && user.name; // safe - stops if user is missing
name || "Anonymous"; // a default (but prefer ?? - see Objects)
```

# The `switch` Statement

---

```
let color = "green";

switch (color) {
  case "red":
    console.log("Stop!");
    break;
  case "green":
    console.log("Go!");
    break;
  default:
    console.log("Unknown color");
}
```

- `case` labels checked top to bottom; `break` stops fall-through
- `default` runs when no case matches

# The Ternary Operator

---

- An **expression** that chooses between two values
- Syntax: `condition ? valueIfTrue : valueIfFalse`

```
let age = 20;  
let message = age >= 18 ? "Adult" : "Minor";  
console.log(message); // "Adult"
```

- Handy for short, simple choices, especially when assigning a value
- Do not nest them; that is what `if / else if` is for

# Guard Clauses Beat Nesting

---

```
// Deeply nested - hard to follow
function pay(user) {
  if (user) {
    if (user.active) {
      if (user.balance > 0) { return "paid"; }
    }
  }
}

// Guard clauses - handle the exits first, then the real work
function pay(user) {
  if (!user) return "no user";
  if (!user.active) return "inactive";
  if (user.balance <= 0) return "no funds";
  return "paid";
}
```